



Microservices

Docker Compose, Multi-Container Apps & Deployment Praxis

Unterrichtseinheit 3/4

Agenda

01

Rückblick UE2

Was haben wir bisher gelernt?

03

Docker Compose

Einführung, Struktur & Befehle

05

Praktische Übungen

Hands-on: Übungen 6–10

02

Warum mehrere Container?

Problemstellung und Motivation

04

Services vernetzen

Netzwerke, Volumes & Ports

06

Best Practices & Debugging

Typische Fehler & Lösungen

Lernziele

Nach dieser Einheit kannst du:



Multi-Container

Anwendungen mit mehreren Containern verstehen und konzipieren



Docker Compose

Compose-Dateien schreiben und Services gemeinsam starten



Netzwerke & Volumes

Persistenz und Service-Kommunikation gezielt einsetzen



Fehler analysieren

Typische Fehler erkennen und systematisch beheben



Rückblick UE2

Das haben wir bereits gelernt

→ Container starten

Mit `docker run` Images als laufende Container instanziiieren

→ Dockerfiles erstellen

Eigene Images mit Dockerfiles beschreiben und konfigurieren

→ Images bauen

Mit `docker build` aus einem Dockerfile ein lauffähiges Image erzeugen

Denkfrage für heute

Was passiert, wenn unsere Anwendung nicht nur einen Container braucht – sondern auch eine Datenbank, einen Cache oder ein Gateway?

→ **Wie starten wir mehrere Container zusammen?**

Problemstellung

Ein einzelner Container reicht für reale Webanwendungen oft nicht aus. Typische Systeme bestehen aus mehreren Komponenten, die zusammenarbeiten:

Frontend

Benutzeroberfläche im Browser

Backend API

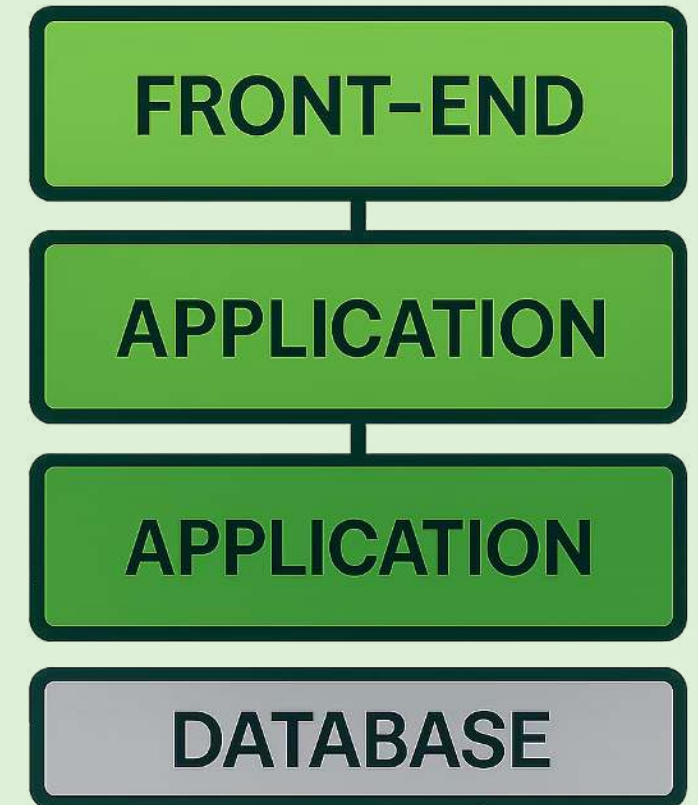
Geschäftslogik & Datenzugriff

Datenbank

Persistente Datenhaltung

Reverse Proxy

Routing & Load Balancing



Denkfrage

Wie würdet ihr 4 Container manuell starten?

Ports merken

Welcher Container hört auf welchem Port?
Konflikte vermeiden.

Reihenfolge beachten

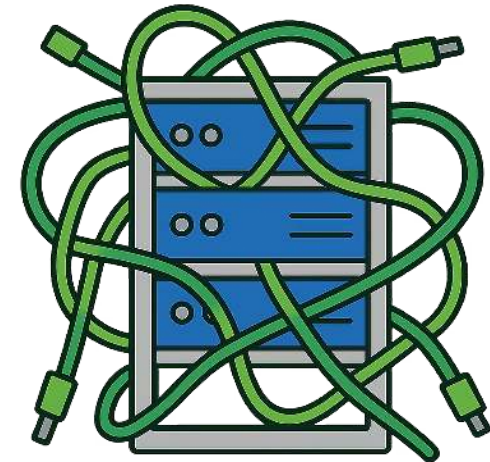
Datenbank muss vor dem Backend gestartet sein.

Netzwerke erstellen

Container müssen sich gegenseitig finden können.



→ Praktisch aufwendig und fehleranfällig — besonders in Teams oder bei häufigen Neustarts. Es braucht eine bessere Lösung.

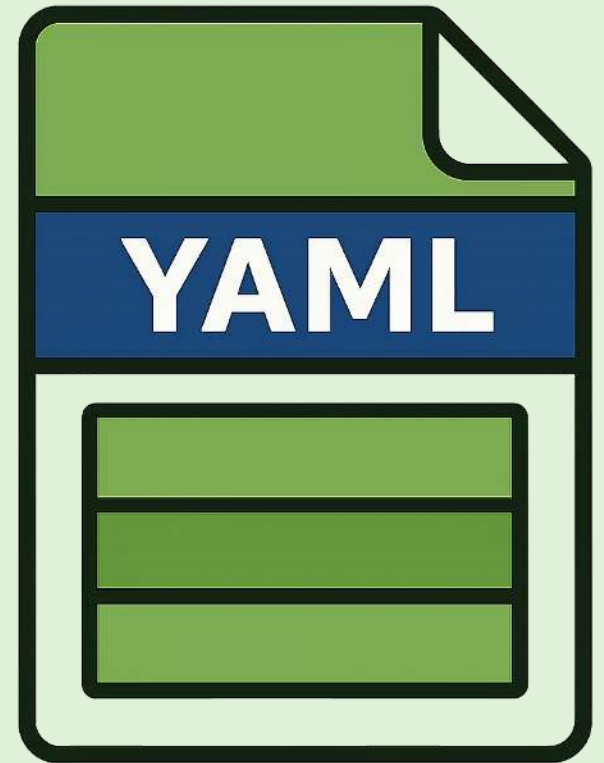


Einführung Docker Compose

Eine Datei. Ein Befehl.

Docker Compose beschreibt alle Container einer Anwendung in einer einzigen Datei – der `compose.yaml`. Statt jeden Container manuell zu starten, genügt ein einziger Befehl, um das gesamte System hochzufahren.

- ❗ Mit `docker compose up` startet ihr alle definierten Services gleichzeitig – inklusive Netzwerk, Volumes und Abhängigkeiten.



Vorteile von Docker Compose



Mehrere Services verwalten

Alle Container einer Anwendung zentral in einer Datei beschreiben und gemeinsam steuern.



Reproduzierbare Umgebungen

Jeder im Team startet exakt dieselbe Umgebung – keine „funktioniert bei mir“-Probleme.



Einfaches Starten & Stoppen

Mit `up` und `down` das gesamte System mit einem Befehl steuern.



Ideal für lokale Entwicklung

Schnell einsatzbereit, ohne komplexe Infrastruktur aufzubauen.

compose.yaml – Grundstruktur

Minimales Beispiel

```
services:  
  web:  
    image: nginx  
  db:  
    image: mysql
```

 Das ist alles, was Docker Compose zum Starten zweier Services benötigt. Kein langes `docker run`-Kommando mit vielen Flags.

Struktur auf einen Blick

services

Der Hauptschlüssel – enthält alle Container-Definitionen der Anwendung.

web / db

Frei wählbare Service-Namen – sie dienen gleichzeitig als Hostnamen im Netzwerk.

image

Gibt an, welches Docker Image verwendet wird. Alternativ: `build: .`

compose.yaml – Erklärung der Schlüssel

1

services: enthält alle Container

Jeder Eintrag unterhalb von `services` entspricht einem Container, der gestartet wird.

2

Jeder Service hat einen eindeutigen Namen

Der Name (z. B. `web`, `db`) ist frei wählbar und dient als interner Hostname im Compose-Netzwerk.

3

Image oder Build

Mit `image:` wird ein fertiges Image genutzt. Mit `build:` wird ein eigenes Dockerfile verwendet.

Erster Start mit Docker Compose

Services starten

```
docker compose up
```

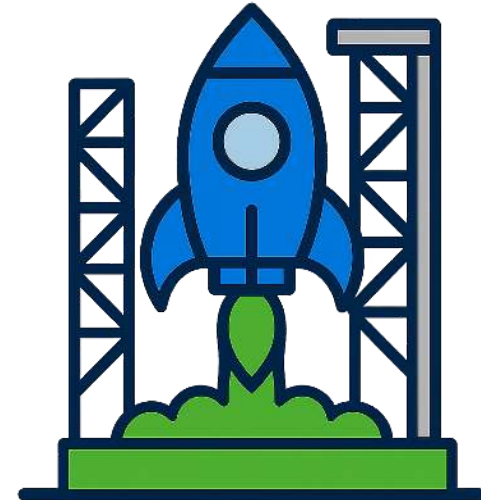
Startet alle Services im Vordergrund – Logs werden direkt ausgegeben.

Im Hintergrund starten

```
docker compose up -d
```

Das Flag `-d` (detached) startet alle Services im Hintergrund. Der Terminal bleibt frei.

- ❑ Beim ersten Start lädt Compose fehlende Images automatisch herunter und erstellt das gemeinsame Netzwerk. Beim nächsten Start geht es schneller.



Erstes Compose Setup

Ziel

Zwei fertige Services mit Docker Compose gemeinsam starten.

Arbeitsordner

uebung-6-erstes-compose-setup

Aufgabe

Nutze die vorbereitete `compose.yaml` in diesem Ordner. Sie enthält:

- nginx
- redis

Starte beide Services mit Compose.

Prüft danach:

- Welche Container laufen?
- Welche Servicenamen habt ihr vergeben?

Minimaler Startpunkt

```
services:  
  web:  
    image: nginx  
  cache:  
    image: redis
```

Hilfreiche Befehle

```
docker compose up -d  
docker compose ps
```

Container prüfen

Compose-spezifisch

```
docker compose ps
```

Zeigt nur die Container des aktuellen Compose-Projekts – mit Status, Ports und Service-Name.

Systemweit

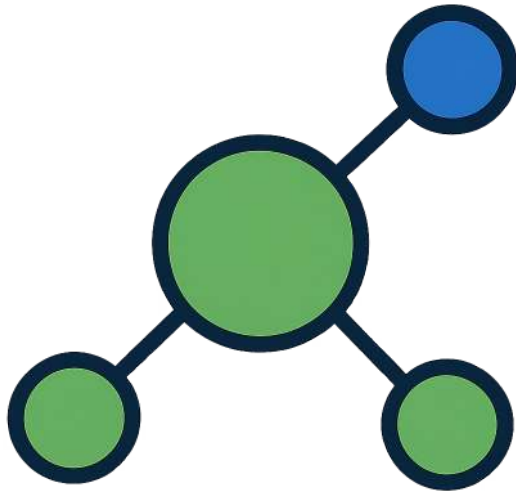
```
docker ps
```

Listet alle laufenden Container auf dem Host – unabhängig vom Compose-Projekt.

 **Tipp:** `docker compose ps` ist der schnellste Weg, um zu prüfen, ob alle Services im erwarteten Zustand sind. Der Status `Up` bedeutet: Container läuft.

 **Exit-Codes:** Ein Status wie `Exit 1` weist auf einen Fehler beim Start hin. Dann helfen `docker compose logs` weiter.

Services kommunizieren



Automatisches Netzwerk

Docker Compose erstellt beim Start automatisch ein gemeinsames Netzwerk für alle Services. Kein manuelles `docker network create` notwendig.

Erreichbarkeit über Service-Namen

Services finden sich gegenseitig über ihren Namen in der `compose.yaml`:

- ⓘ Der Service-Name ist gleichzeitig der DNS-Hostname innerhalb des Netzwerks.

db

backend

redis

Beispiel: Microservice Setup

compose.yaml

```
services:
  api:
    build: .
  db:
    image: postgres
```

Interne Kommunikation

Die `api` kann die Datenbank direkt über den Namen `db` ansprechen – z. B. als Connection-String:

```
postgres://db:5432/mydb
```

 Kein IP-Management nötig. Docker löst den Namen `db` automatisch auf die richtige Container-IP auf.

Ports definieren

Syntax in der compose.yaml

```
ports:  
  - "8080:80"
```

Format: "HOST_PORT:CONTAINER_PORT"

Was passiert dabei?

1

Host: Port 8080

Im Browser: localhost:8080

2

Container: Port 80

nginx lauscht intern auf Port 80



Ohne Portfreigabe ist der Service nur intern im Compose-Netzwerk erreichbar – nicht vom Host-Browser aus.

Web-Service erreichbar machen

Ziel

Ein Compose-Service mit Portfreigabe im Browser erreichen und verstehen.

Arbeitsordner

uebung-7-web-service-erreichbar

Aufgabe

1. Starte das vorbereitete Setup.
2. Öffne die `compose.yaml` und identifiziere die Portfreigabe des Service `web`.
3. Öffne `http://localhost:8080` im Browser.
4. Prüfe mit `docker compose ps`, welcher Port veröffentlicht wurde.

Startpunkt

```
ports:  
- "8080:80"
```

Volumes verstehen



Was sind Volumes?

Volumes speichern Daten **außerhalb des Containers** dauerhaft auf dem Host. Sie überleben auch dann, wenn der Container gestoppt oder gelöscht wird.

Wichtig für:



Datenbanken



Logs



Uploads

Beispiel: Volume in compose.yaml

Vollständiges Beispiel

```
services:
  db:
    image: postgres
    volumes:
      - dbdata:/var/lib/postgresql/data

volumes:
  dbdata:
```

Erklärung

volumes: im Service

Verknüpft das benannte Volume `dbdata` mit dem Pfad im Container.

volumes: auf oberster Ebene

Deklariert das Volume als globale Ressource des Compose-Projekts.

-  Benannte Volumes werden von Docker verwaltet – kein manueller Pfad auf dem Host notwendig.

Warum Volumes wichtig sind

Ohne Volume

Alle Daten liegen im Container-Dateisystem. Wird der Container gelöscht (`docker compose down`), sind **alle Daten unwiederbringlich verloren**.

Mit Volume

Daten liegen auf einem persistenten Volume außerhalb des Containers. Nach einem Neustart oder Rebuild sind **alle Daten weiterhin vorhanden**.

 Besonders kritisch bei Datenbanken: Ohne Volume verliert ihr bei jedem `docker compose down` sämtliche gespeicherten Datenbankeinträge.

Volume einbauen und verstehen

Ziel

Nachvollziehen, warum Daten oder Dateien mit Volumes erhalten bleiben.

Arbeitsordner

uebung-8-volume-verstehen

Aufgabe

1. Starte das vorbereitete Setup mit benanntem Volume.
2. Prüfe mit `docker volume ls`, welches Volume angelegt wurde.
3. Stoppe es mit `docker compose down`.
4. Prüfe danach erneut, warum das Volume weiterhin vorhanden ist.

Reflexionsfrage

Warum ist das besonders für Datenbanken wichtig?

Startpunkt

```
services:
  db:
    image: postgres
volumes:
  - dbdata:/var/lib/postgresql/data

volumes:
  dbdata:
```

Build statt Image

Eigenes Dockerfile nutzen

```
services:  
  app:  
    build: .
```

Anstelle eines fertigen Images aus Docker Hub kann Compose ein eigenes Dockerfile bauen. Der Punkt (.) verweist auf das aktuelle Verzeichnis.

Wann **build** statt **image**?

image:

Fertiges Image von Docker Hub oder einer Registry verwenden

build:

Eigenen Code einbetten und ein individuelles Image bauen

- ⓘ Mit `docker compose build` kann das Image auch manuell neu gebaut werden, ohne die Services zu starten.

Eigenes Dockerfile mit Compose nutzen

Ziel

Ein eigenes Image per `build` zusammen mit einem zweiten Service starten.

Arbeitsordner

`uebung-9-dockerfile-mit-compose`

Aufgabe

Nutze das vorbereitete App-Beispiel in diesem Ordner. Die vorbereitete `compose.yaml` enthält:

- `app`
- `redis`

Starte beide Services.

Prüft danach:

- Wird `app` gebaut oder nur heruntergeladen?
- Welche Container laufen?
- Was seht ihr in den Logs?
- Ist die App unter `http://localhost:5001` erreichbar?

📌 Mit `build:` erstellt Compose beim ersten Start automatisch ein Image aus dem lokalen Dockerfile.

Logs ansehen

Alle Services

```
docker compose logs
```

Gibt die Logs aller Services des aktuellen Projekts aus.

Einzelner Service

```
docker compose logs app
```

Filtert die Ausgabe auf den angegebenen Service.



Tipp: Live-Logs

Mit dem Flag `-f` (follow) werden Logs in Echtzeit angezeigt:

```
docker compose logs -f app
```



Logs sind die erste Anlaufstelle bei Fehlern. Fehlermeldungen, Stack Traces und Verbindungsprobleme erscheinen hier zuerst.

Stoppen & Entfernen

Services stoppen & Container entfernen

```
docker compose down
```

Stoppt alle Services und entfernt die Container sowie das automatisch erstellte Netzwerk. Volumes bleiben erhalten.

Inkl. Volumes löschen

```
docker compose down -v
```

Entfernt zusätzlich alle im Projekt deklarierten Volumes – Daten werden unwiderruflich gelöscht.



Vorsicht mit

Typische Fehler

Port bereits belegt

Ein anderer Prozess nutzt denselben Host-Port.

Falscher Service-Name

Tippfehler im Service-Namen führen zu DNS-Fehlern.

Build schlägt fehl

Fehler im Dockerfile oder fehlende Dateien.

Neustart-Schleife

Container crasht und wird ständig neu gestartet.

Env-Variablen fehlen

Konfiguration fehlt → Anwendung kann nicht starten.

Volume falsch gemountet

Pfad stimmt nicht → Daten nicht sichtbar.

Fehler erkennen: Erste Analyse

Wichtige Diagnosebefehle

```
docker compose ps
docker compose logs
docker ps
```

Was genau prüfen?

→ Läuft der Container?

Status in `docker compose ps` prüfen

→ Neustart-Schleife?

Restarting in der Status-Spalte

→ Exit-Status?

Exit 1 oder höher weist auf einen Fehler hin

→ Fehlermeldungen?

Logs mit `docker compose logs [service]` lesen

Fehler: Port bereits belegt

Fehlermeldung erkennen

```
Bind for 0.0.0.0:8080 failed:  
port is already allocated
```

Lösung



Anderen Port wählen

In der `compose.yaml` einen freien Host-Port eintragen, z. B. `"8081:80"`



Blockierenden Prozess stoppen

Mit `lsof -i :8080` (macOS/Linux) oder dem Task-Manager (Windows) den Prozess finden und beenden.

Fehler: Falscher Service-Name

Fehlermeldung erkennen

```
could not translate host name  
"database" to address:  
Name or service not known
```

Lösung

Der Connection-String in der App muss exakt den Service-Namen aus der `compose.yaml` verwenden:

`db`

`redis`

`api`



Tipfehler wie `Database` statt `db` reichen aus, damit der DNS-Name nicht aufgelöst werden kann.

Fehler: Build schlägt fehl

Build manuell ausführen

```
docker compose build
```

Zeigt detaillierte Ausgabe des Build-Prozesses – inklusive der Zeile, an der der Fehler auftritt.

Typische Ursachen

Datei fehlt

Eine per `COPY` referenzierte Datei ist nicht vorhanden.

`COPY` falsch

Quell- oder Zielpfad im `COPY`-Befehl ist fehlerhaft.

Syntaxfehler im Dockerfile

Ungültige Instruktionen oder Tippfehler in Schlüsselwörtern.

Fehler: Container startet ständig neu

Erkennen

NAME	STATUS
app	Restarting (1) ...

Der Status `Restarting` in `docker compose ps` zeigt eine Neustart-Schleife an. Der Exit-Code gibt einen ersten Hinweis auf die Ursache.

Lösung: Logs lesen

```
docker compose logs app
```

- Die Logs enthalten fast immer den eigentlichen Fehler, der den Container zum Absturz bringt – z. B. eine fehlende Konfiguration, ein Connection-Fehler oder ein Programm-Crash.

Fehler: Environment-Variablen fehlen

Erkennen

```
Error: Missing required  
environment variable:  
DATABASE_URL
```

Lösung

Umgebungsvariablen direkt in der `compose.yaml` setzen:

```
environment:  
  DATABASE_URL: postgres://db:5432/mydb  
  SECRET_KEY: supersecret
```

- 📄 Für sensible Daten wie Passwörter alternativ eine `.env`-Datei nutzen und diese nicht in Git einchecken.

Fehler: Volume-Probleme

Erkennen

Typische Symptome:

- Daten fehlen nach Neustart
- Dateien im Container nicht sichtbar
- Datenbank startet leer

Lösung: Volumes prüfen

```
docker volume ls
```

Zeigt alle vorhandenen Volumes. Prüft, ob das erwartete Volume tatsächlich erstellt wurde.

- ❗ Volumes müssen sowohl im Service als auch auf der obersten Ebene der `compose.yaml` deklariert sein.

In Container hineinsehen

Manchmal startet der Container, aber Dateien fehlen.

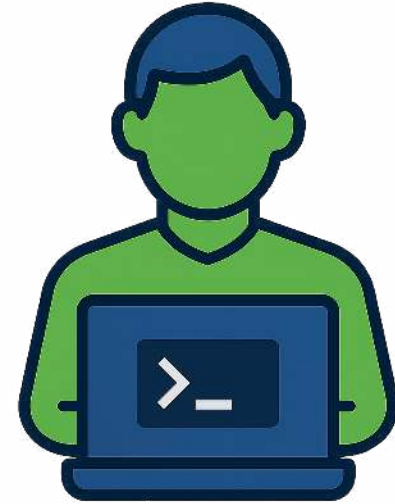
Dann direkt in den laufenden Container wechseln und das Dateisystem von innen inspizieren:

Mit Shell (Alpine/minimale Images)

```
docker compose exec app sh
```

Mit Bash (Ubuntu/Debian-basierte Images)

```
docker compose exec app bash
```



 Nach dem Login befindet ihr euch direkt im laufenden Container. Alle Befehle werden dort ausgeführt – ihr seht das echte Dateisystem zur Laufzeit.

Was im Container prüfen?

Nach dem Login ausführen:

```
pwd  
ls -la  
cd /app  
ls -la
```

Was prüft ihr damit?

→ Sind Dateien vorhanden?

Prüfen, ob der erwartete Inhalt im Arbeitsverzeichnis liegt.

→ Wurde **COPY ..** korrekt ausgeführt?

Fehlende Dateien deuten auf einen COPY-Fehler im Dockerfile hin.

→ Liegt Code im richtigen Ordner?

Der **WORKDIR** im Dockerfile muss mit dem Pfad übereinstimmen.

Beispiel: Debugging Dateisystem

Szenario: App startet nicht

Im Container den Inhalt prüfen:

```
ls -la /app  
cat package.json
```

Mögliche Ursachen

package.json fehlt

Die Datei wurde nicht ins Image kopiert – COPY prüfen.

Falscher WORKDIR

Das Arbeitsverzeichnis stimmt nicht mit dem Pfad überein.

Falscher COPY-Pfad

Quell- oder Zielpfad im Dockerfile fehlerhaft angegeben.

Weitere hilfreiche Container-Befehle

Im Container ausführen:

```
env  
whoami  
ps aux
```

Was prüft ihr damit?

env

Alle gesetzten Umgebungsvariablen anzeigen – sind alle notwendigen Werte vorhanden?

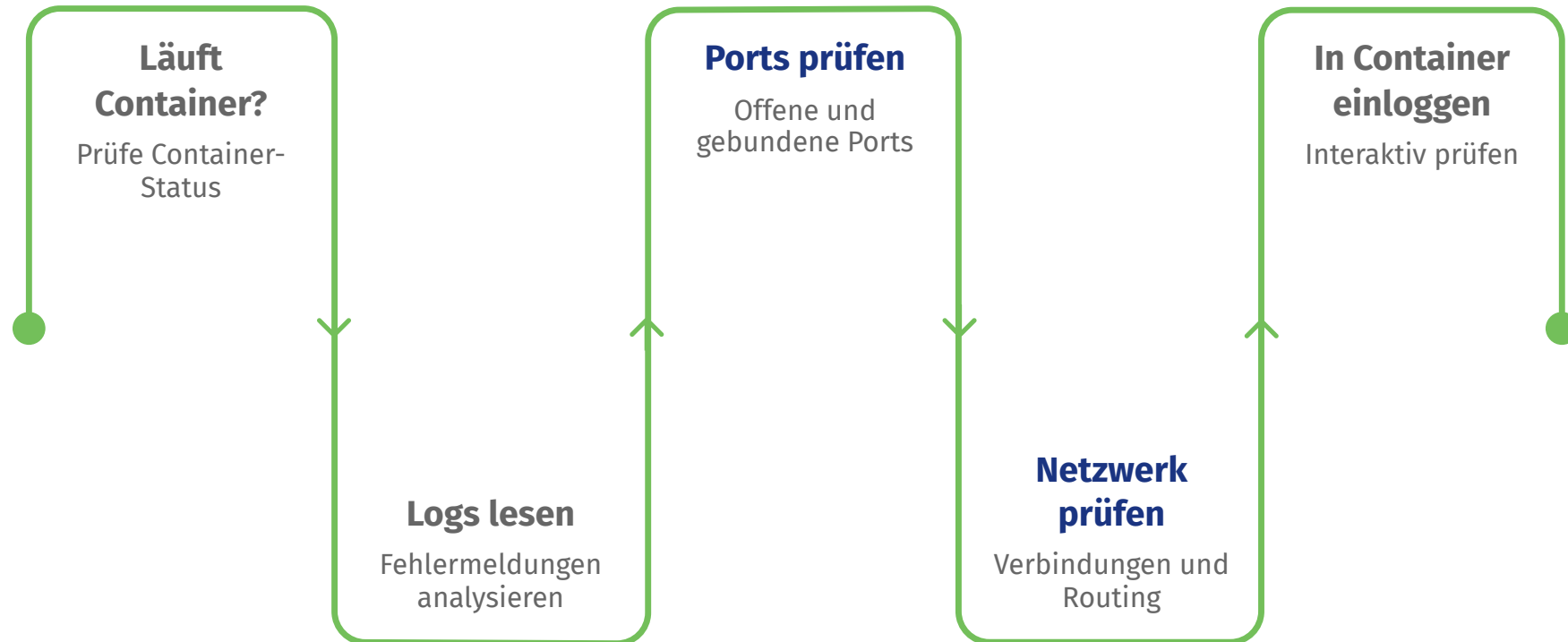
whoami

Unter welchem User läuft der Prozess? Root oder ein eingeschränkter Benutzer?

ps aux

Läuft der erwartete Prozess (z. B. Node, Python, nginx) tatsächlich?

Debugging-Reihenfolge



Diese Reihenfolge führt systematisch von der groben Übersicht zur detaillierten Ursache. Beginnt immer von oben – die meisten Fehler lassen sich schon in Schritt 1–3 identifizieren.

Compose Setup debuggen

Ziel

Ein fehlerhaftes Compose Setup systematisch analysieren.

Arbeitsordner

uebung-10-compose-debugging

Szenario

Der Container läuft, aber unter `http://localhost:8083` erscheint nicht die erwartete Startseite.

Aufgabe

Welche Schritte würdet ihr in der richtigen Reihenfolge machen?

- Logs?
- Exec?
- Dateien prüfen?

→ Antwort im Chat

📌 Geht systematisch vor – die Debugging-Reihenfolge aus Slide 38 ist euer Leitfaden.

Musterlösung: Debugging

Schritt-für-Schritt

```
docker compose logs web
```

```
docker compose exec web sh
```

```
ls -la /usr/share/nginx/html
```

Was prüfen?



Liegt index.html dort?

nginx erwartet die Startseite unter `/usr/share/nginx/html/`



Wurde korrekt kopiert?

Den `COPY`-Befehl im Dockerfile auf Quell- und Zielpfad prüfen.



Liegt `index.html` im richtigen Pfad, zeigt nginx sie automatisch an. Fehlt sie, gibt nginx eine 403- oder leere Seite zurück.

Transfer: Compose systematisch debuggen

📋 Szenario: Ein Service erreicht die Datenbank nicht – der Container läuft aber.

1

Logs prüfen

`docker compose logs api`

2

Netzwerk prüfen

Sind beide Services im selben Compose-Projekt?

3

Service-Namen prüfen

Stimmt der Hostname im Connection-String?

4

Config prüfen

Env-Variablen, Ports und Volume-Pfade validieren

Compose für Microservices

Realistische Microservice-Architektur



Vorteile dieses Ansatzes

Mit einer einzigen `compose.yaml` lässt sich ein vollständiges Microservice-System lokal abbilden:

Klare Trennung

Jeder Service hat eine eigene Verantwortung und kann unabhängig entwickelt werden.

Zentrales Gateway

Ein Reverse Proxy (z. B. nginx oder Traefik) verteilt Requests an die richtigen Services.

Lokale Entwicklung

Das gesamte System mit einem einzigen Befehl starten und testen.

Grenzen von Docker Compose

✓ Gut geeignet für

- Lokale Entwicklung & Prototypen
- Automatisierte Tests
- Kleine Systeme mit wenigen Services
- CI/CD-Pipelines (einfache Szenarien)

✗ Nicht geeignet für

- Produktionssysteme mit hohen Anforderungen
- Automatische Skalierung über mehrere Hosts
- Self-Healing und automatische Neuverteilung
- Komplexe Orchestrierung großer Cluster

❗ Compose ist ein hervorragendes Lernwerkzeug und Einstieg in die Container-Welt – für wachsende Produktionssysteme braucht es jedoch Kubernetes.



Übergang zu Kubernetes

Wenn Systeme wachsen und Compose an seine Grenzen stößt, ist Kubernetes der nächste Schritt:



Skalierung

Services horizontal auf viele Instanzen skalieren



Self-Healing

Ausgefallene Container werden automatisch neu gestartet



Orchestrierung

Intelligente Verteilung von Workloads auf Nodes

Diskussion



Eure Erfahrungen aus den Übungen

Welche Debugging-Methode wirkt am hilfreichsten?

Logs

`docker compose logs` –
Fehlermeldungen direkt sehen

Exec

`docker compose exec` – direkt im
Container nachschauen

 → Antworten im Chat

Zusammenfassung

Das haben wir heute erarbeitet:



Docker Compose

Mehrere Services in einer `compose.yaml` beschreiben und mit einem Befehl starten



Debugging mit Logs

Mit `docker compose logs` Fehler schnell identifizieren und analysieren



Multi-Container Systeme

Services kommunizieren über automatisch erstellte Netzwerke per Service-Namen



Debugging mit Exec

Mit `docker compose exec` direkt in den Container einloggen und Dateisystem prüfen

Ausblick: UE4

Beim nächsten Mal steigen wir von Docker Compose in die Welt der Container-Orchestrierung ein:

Kubernetes

Der Industriestandard für Container-Orchestrierung

Skalierung

Services dynamisch hoch- und runterskalieren

Load Balancing

Traffic intelligent auf Instanzen verteilen

CI/CD Deployments

Automatisierte Pipelines für schnelle Releases

